

CONSTRUCTION OS

PRD_14

Platform Architecture Constitution
and Engineering Guardrails

Master implementation rules that make PRD_1 through PRD_13 operate as one exceptional Construction OS

Version	1.0
Date	3 August 2026
Status	Confirmed Engineering Direction - Full-Stack Team Charter
Audience	Senior Full-Stack Engineering, Product, QA, DevOps, Security and Data Teams

Compatibility rule

This PRD extends the Construction OS master product requirements and all earlier PRDs. Earlier requirements remain authoritative unless this document explicitly amends them.

Contents

- 1 Executive Directive
- 2 Review of PRD_1 through PRD_13
- 3 Requirement Precedence and Decision Control
- 4 Recommended Architecture Baseline
- 5 Shared Platform Engines
- 6 Domain Boundaries and Ownership
- 7 Cross-Module Integration Contract
- 8 Data and Relationship Standards
- 9 Authorization, Tenant Isolation and Privileged Access
- 10 Role-Specific UX Without Product Fragmentation
- 11 Company-Specific Features Without Core Forks
- 12 Financial, Document and Record Truth Rules
- 13 Reliability, Observability and Performance
- 14 Delivery Order and Engineering Workflow
- 15 Definition of Done and Test Gates
- 16 Known Supersessions and Critical Risks
- 17 Required Deliverables from the Full-Stack Team
- 18 Acceptance Criteria

1. Executive Directive

Construction OS must be developed as one connected operating system, not as a collection of independent dashboards or modules. The product may be broad and role-specific, but its permissions, workflows, documents, tasks, comments, notifications, audit history, search, calendar and data relationships must come from shared platform services.

Non-negotiable product rule

The platform may look different to a Finance user, Architect, HR user, Project Manager, Company Admin or Platform Admin, but the underlying records, status logic, security rules and audit model must remain unified.

The main risk identified across the existing PRDs is not excessive ambition. It is duplicate implementation: separate permission logic, separate comment systems, separate approval rules, separate payment summaries or separate document histories inside each module. That would make the product expensive, inconsistent and difficult to trust.

2. Review of PRD_1 through PRD_13

PRD group	What it establishes	Engineering implication
PRD_1-2	Master SaaS vision, enterprise modules, NAV-style Role Centers, cards, lists, posting, ledgers, APIs, events and delivery roadmap.	Use these as the platform baseline and reusable interaction/data patterns.
PRD_3-5	Comment-to-task conversion, cross-department task orchestration and correct active-role navigation.	Use one comment/mention service, one task engine and explicit role context.

PRD group	What it establishes	Engineering implication
PRD_6-8	Public professional/contractor profiles, Platform Admin governance, corporate groups, access control and data governance.	Identity, tenancy and authorization must be foundational services, not module code.
PRD_9-10	Private user calendars, availability, external calendar sync, adaptive role interfaces and company-wide project visibility with restricted sections.	Separate personal visibility from shared availability and use section-level authorization inside a universal project shell.
PRD_11	Super Platform Admin, Platform Admin actions, tenant lifecycle, support access, custom features, staged releases and undo.	Platform operations require a separate control plane with versioned, reversible administration.
PRD_12-13	Clients, purchased units, contract-linked tasks, contractor/client financial summaries and detailed payments.	Client, contractor, contract, unit, task and payment records must be relationally connected and permission-filtered.

Overall review result

The existing PRDs already contain the elements of an exceptional product. The missing layer is an explicit engineering constitution that prevents every new PRD from becoming a new technical subsystem.

3. Requirement Precedence and Decision Control

1. The latest explicit Product Owner decision is authoritative.
2. An explicit amendment or supersession statement overrides the earlier rule.
3. Where two PRDs conflict without an explicit amendment, the later-numbered PRD takes precedence.
4. Where a later PRD is silent, the earlier Master PRD and related PRDs remain valid.
5. A deferred or unanswered question must remain configurable or blocked from implementation; developers must not silently choose a business rule.

ARCH-REQ-001 Central requirements register

Maintain one version-controlled register containing requirement ID, source PRD, current wording, superseded requirement, affected service, frontend surface, database entities, API/event contracts, test cases and release status.

ARCH-REQ-002 Architecture Decision Records

Every major technical decision must be documented as an ADR, including alternatives, reason, consequences, rollback impact and the PRDs it supports.

ARCH-REQ-003 No hidden interpretation

When a requirement is ambiguous or technically unsafe, the team must raise a product decision rather than implement an assumption that becomes permanent behavior.

4. Recommended Architecture Baseline

Recommended launch architecture

Start with a modular monolith using strict domain boundaries, a relational transactional database, object storage, search indexing, background jobs, an outbox/event mechanism and independently deployable frontend modules. Extract services only when scale, isolation or release independence justifies it.

- Do not begin with many microservices. The product has complex transactions and rapidly evolving business rules; premature service separation will increase inconsistency and operational cost.
- Keep domain modules isolated in code, database access and ownership even when deployed together.
- No module may read or write another module's internal tables directly. Interaction occurs through application services, stable APIs or domain events.
- Use one web application shell with role-aware navigation and independently owned feature modules.

- Use an event/outbox pattern for reliable cross-module propagation, notifications, search indexing, aggregates and external integrations.
- Design tenant boundaries so standard customers can use shared infrastructure while enterprise tenants can later move to dedicated databases without changing domain contracts.

Layer	Responsibility
Control plane	Super Platform Admin, Platform Admin, tenant lifecycle, subscriptions, feature versions, releases, support access, backups and platform health.
Identity and policy plane	Accounts, company membership, active role, group context, permissions, support-mode claims and authorization decisions.
Domain application plane	Projects, tasks, documents, contracts, finance, clients, contractors, HR, procurement, inventory and site operations.
Integration plane	Outbox/events, external calendars, email/WhatsApp, cloud storage, accounting integrations and webhooks.
Data and evidence plane	Transactional records, immutable audit history, document/object storage, search indexes, analytics projections and backups.

5. Shared Platform Engines

The following capabilities must be implemented once and consumed by every module. A module may configure these engines but may not create an alternative implementation.

Shared engine	Mandatory responsibility
Identity and Tenant Context	Resolve user, tenant, company, corporate group, active role, impersonation/support mode and session context for every request.
Authorization Policy Engine	Deny by default; evaluate tenant, company, module, record, field and action permissions; support grants, revocations, inheritance and sensitive sections.
Workflow and Approval Engine	Reusable state machines, approval chains, rejection, override, delegation, conditions and transition validation.
Task and Action Engine	Assignments, dependencies, due dates, status, linked records, comment-generated tasks, review/closure and immutable timeline.
Comment, Mention and Collaboration Engine	Threads, @mentions, attachments, visibility, linked tasks, notifications and cross-record references.
Document and Revision Engine	Controlled files, revisions, official/superseded status, signatures, access, retention, recycle rules and evidence links.
Audit and Timeline Engine	Immutable actor/action/before-after evidence with user-facing projections for record timelines and security investigation.
Notification Engine	In-app, email, WhatsApp and calendar reminders from shared events, user preferences, critical rules and deduplication.
Search and Access-Request Engine	Permission-aware indexing, locked metadata, access requests and zero leakage of restricted fields.
Calendar and Scheduling Engine	Private personal calendar, team availability, reminders, external synchronization and record deep links.
Feature and Configuration Engine	Tenant feature flags, role shells, company configuration, custom-feature versions, staged rollout and rollback.
Reporting and Aggregate Engine	Permission-filtered summaries, financial/profile aggregates, scheduled reports and traceable source versions.

ARCH-CORE-001 Shared service enforcement

Code review and architecture tests must prevent modules from creating duplicate permission, comment, audit, notification, document, approval or task frameworks.

6. Domain Boundaries and Ownership

Domain	Owns	Must not own
Projects	Project identity, participation, progress context, project relationships and project shell.	Contract internals, HR records, payment records or document binaries.
Tasks	Task state, assignment, dependencies, contributions, linked-object references and task timeline.	The business record that originated the task.
Contracts/Legal	Contract identity, parties, versions, clauses, approvals, legal status and official documents.	Client identity, contractor profile or payment transaction data.
Finance	Approved financial summaries, invoices, payments, allocations, approvals and reconciliation.	Contract legal content or project permissions.
Sales/Clients/Units	Client identity, purchased-unit relationship, sales status and unit-facing commercial context.	Architectural master drawings or finance ledger internals.
Contractors/Procurement	Contractor/supplier profile, qualifications, procurement relationships and performance.	Finance payment truth or legal contract versions.
Architecture/Engineering	Unit design, floor plans, drawings, revisions and technical deliverables.	Client financial data or contract approval authority.
HR	Employee profile, attendance, leave, schedules and private HR records.	General project task records or company finance.
Platform Administration	Tenant/control-plane records, features, releases, recovery, support grants and technical metadata.	Company operational data unless support access is active.

Ownership rule

A record has one authoritative domain owner. Other modules reference it by permanent ID and receive permitted projections; they do not copy and independently edit the same business fact.

7. Cross-Module Integration Contract

- Use stable APIs for direct user-initiated commands and queries.
- Use domain events for asynchronous propagation such as notifications, search indexing, aggregates, calendar entries and related timelines.
- Use an outbox and idempotent consumers so retries cannot duplicate tasks, payments, notifications or audit records.
- Every request and event must carry tenant ID, company ID, actor/effective-user ID, active role, correlation ID and source record ID.
- Cross-module failures must not silently produce partial state. Use transactions within a domain and compensating workflows across domains.
- Do not expose internal database IDs or unrestricted payloads to external integrations.

Example interaction	Correct implementation
Finance tags Architect in a client contract comment	Comment service stores the thread; Task engine creates a linked Architect task; Contract, Client and Unit remain referenced by ID; Finance receives review/closure state.
HR approves leave	HR owns the leave record; Calendar receives an availability projection; no task/deadline mutation occurs because PRD_9 explicitly forbids automatic action.
Contractor payment is recorded	Finance stores the payment and proof; reconciliation/aggregate projections update; contractor profile reads permitted summaries rather than owning payment data.
Project opens for every company member	Project service returns the common project shell; authorization service filters each section and nested record.

Example interaction	Correct implementation
Platform Admin deploys a tenant feature	Control plane changes a versioned feature assignment; deployment emits events; tenant module loads the feature through the extension contract.

8. Data and Relationship Standards

Every important business entity must be addressable, traceable and linkable across the platform without copying its data.

Mandatory attribute	Rule
Permanent ID	Stable UUID/ULID or equivalent; never reused.
Tenant and company scope	Required on every operational record and enforced by database/application policies.
Project scope	Required where the record belongs to a project; optional only for legitimate company-level records.
Owner/responsible party	Record owner, department and assigned actors stored explicitly.
Status and version	Central status catalog or domain state machine; optimistic concurrency prevents lost updates.
Actor and timestamps	Created/updated by, created/updated at, effective role and impersonation/support identity.
Archive/deletion state	Soft deletion/recycle/retention rules according to domain; audit history never silently removed.
Relationship links	Typed links to project, task, contract, document, client, contractor, unit, payment or other source records.
Source and authority	Identify whether a value is entered, imported, calculated, approved or projected.

ARCH-DATA-001 No orphan links

Foreign keys or validated relationship services must prevent links to another tenant, wrong company, deleted source or incompatible record type.

ARCH-DATA-002 Historical reproducibility

Versioned records and aggregates must allow the platform to explain which approved values and source records produced a past result.

9. Authorization, Tenant Isolation and Privileged Access

- Authorization is enforced on the backend for every list, record, field, download, search result, report, export and event subscription.
- Frontend hiding is a usability measure, not a security boundary.
- Company and corporate-group context must be explicit; never infer tenant scope only from a URL or client payload.
- Active role context must be validated against current memberships and included in authorization decisions.
- Sensitive HR, Finance, Legal and contract sections use field/section-level permissions in addition to project visibility.
- Admin impersonation and Platform Admin support mode must show a persistent visual banner and record both the acting identity and effective company user/context.
- Because PRD_11 allows support access without automatic expiry, the company must always have a visible support-access indicator and immediate revoke control.
- Search indexes, analytics projections, notifications and caches must apply the same permissions as primary records.

Security rule

No user, Admin, Platform Admin, integration or background worker receives broader data access merely because it operates at a higher technical layer. Every action requires an explicit policy decision and auditable context.

10. Role-Specific UX Without Product Fragmentation

- Role-specific themes may substantially change navigation, dashboard composition and emphasis as confirmed in PRD_9.
- Role-specific shells must reuse the same design system, record components, status language, validation, deep links and APIs.
- Do not build separate Finance, Architect or HR applications with copied business logic.
- A multi-role user switching active role changes shell and navigation, not identity or the underlying record truth.
- Use progressive disclosure: each user sees the work, fields and actions relevant to the active role while the platform remains broad behind the interface.
- The same record should open from task, search, calendar, notification, project, client or contractor contexts through canonical routes and permission-aware views.
- Platform Default, Night Mode and Creative User Type Theme must remain presentation concerns; they cannot change permissions, statuses or workflow outcomes.

ARCH-UX-001 One component contract

Core record headers, timelines, comments, files, permissions, status controls and related-record panels must use shared components and data contracts across role interfaces.

11. Company-Specific Features Without Core Forks

PRD_11 permits completely custom tenant features. This must be implemented as a controlled extension system, not as copied code or permanent customer branches.

Required extension property	Implementation rule
Tenant isolation	Feature assignment and data are scoped to approved tenants; other tenants cannot discover routes, schemas or data.
Versioning	Every feature release has a semantic version, migration version, compatibility range and change log.
Feature flag/manifest	A signed or trusted manifest declares routes, permissions, events, settings, dependencies and rollback version.
Database changes	Use namespaced, repeatable migrations; never manually alter one tenant database without a migration record.
Testing	Core regression, tenant isolation, permission, migration and rollback tests are mandatory before deployment.
Observability	Errors, performance and adoption are measurable separately for the tenant feature.
Promotion to core	Generalize behavior and remove tenant-specific confidential logic before offering it platform-wide.
Rollback	Disable/restore the prior version without affecting unrelated tenant data or core features.

Forbidden approach

Do not create a separate source-code fork, copied module, duplicated database schema or one-off permission system for a company. Those choices would destroy maintainability and make global updates unsafe.

12. Financial, Document and Record Truth Rules

12.1 Finance-approved manual values

- For PRD_13, the latest Finance-approved manual contract value, paid/received value and remaining payable/receivable value are the official profile summary.

- Detailed payment records are supporting financial evidence and are individually traceable to date, amount, reference, method and proof document.
- The system calculates comparison values and displays structured reconciliation warnings but never silently overwrites the Finance-approved manual summary.
- Consolidated client/contractor totals use the latest approved manual summaries and remain grouped by currency unless an approved conversion policy exists.
- Every manual value must record source, approver, version, previous value and reconciliation state.

12.2 Official documents and revisions

- The Document engine owns file identity, revision lineage, official/superseded state, signatures, retention and permission checks.
- Tasks, contracts, units, projects and comments link to document versions; they do not store uncontrolled duplicate files as the official record.
- Deleting or replacing a displayed file must follow the relevant domain's confirmed recycle, retention and audit rules.

12.3 User-facing timelines

Audit evidence, domain events and user-facing timelines should originate from the same committed business action but may have different projections. Do not create three unrelated histories that can disagree.

13. Reliability, Observability and Performance

- Every request and background job uses correlation IDs and structured logs containing tenant/company context without exposing sensitive payloads.
- Monitor permission denials, tenant-boundary violations, failed events, duplicate suppression, queue delay, search-index lag, document processing, external sync and custom-feature health.
- Use optimistic locking/version checks for records edited by multiple users.
- Use idempotency keys for payment, posting, approval, task generation, recovery, deployment, rollback and destructive operations.
- Use backups, point-in-time recovery and tested tenant restore procedures; a backup that has not been restoration-tested is not considered valid.
- Define service-level objectives for login, project shell, common lists, search, uploads, notification delivery and background propagation before production launch.
- Design degraded modes: a notification or external-calendar outage must not corrupt the authoritative internal record.

13.1 Super Platform Admin Undo

PRD_11 requires Super Platform Admin review and undo. Each privileged action must declare one of four reversibility classes before execution:

Class	Meaning
Exact reversible	Restore the prior version automatically, such as feature flags, configuration or a deployment version.
Compensatable	Create a new opposite action while preserving history, such as permission grant/revocation or posted corrections.
Backup-required	Restoration requires a tenant-scoped backup/recovery workflow.
Irreversible	Cannot be safely undone; UI must warn before execution and must not present a misleading instant Undo control.

14. Delivery Order and Engineering Workflow

Build order rule

Do not build complete business modules first and attempt to connect them later. Build the shared platform foundation, then release vertical workflows that use it end to end.

Stage	Required output before proceeding
A. Architecture foundation	ADR pack, C4 diagrams, tenant model, domain map, deployment model, security model, design system and repository standards.
B. Shared platform core	Identity/context, authorization, audit, events/outbox, comments, tasks, documents, notifications, search, workflow and feature configuration.
C. First vertical workflow	Project -> task -> comment -> document -> approval -> notification -> audit, tested across roles and tenants.
D. Commercial/legal workflow	Client/contract/unit and contractor/contract relationships with section permissions and canonical records.
E. Financial workflow	Finance-approved summaries, detailed payments, proof, reconciliation, permissions and aggregates.
F. HR/calendar workflow	Leave approval, private calendar, team availability, reminders and external synchronization.
G. Platform control plane	Tenant lifecycle, recovery, support mode, feature deployment, staged release, oversight and undo classes.
H. Scale and premium modules	Offline, BIM/Digital Twin, advanced analytics, dedicated databases and enterprise integrations.

- Each stage must deliver usable vertical behavior, not only database tables or isolated screens.
- Frontend, backend, QA, security, data and DevOps work from the same requirement IDs and acceptance criteria.
- Every pull request references requirement IDs and includes relevant tests, migrations and audit/event effects.
- Feature completion requires demo with realistic roles and cross-module records, not a developer-only happy path.

15. Definition of Done and Test Gates

Gate	Mandatory proof
Functional	All mapped acceptance criteria pass; no deferred rule is implemented as an assumption.
Authorization	Tenant-isolation, role, section, field, download, search, export and background-worker tests pass.
Data integrity	Relationship, concurrency, idempotency, migration, rollback and reconciliation tests pass.
Auditability	Actor, effective user, active role, before/after, record links and correlation ID are verified.
Cross-module	Events arrive once effectively, linked records remain canonical and partial failures are recoverable.
UX	Role shell, responsive behavior, accessibility, loading/empty/error/denied states and canonical deep links pass.
Security	MFA/high-risk re-auth, secret handling, upload scanning, dependency scanning and penetration tests pass.
Operations	Monitoring, alerts, backup/restore, feature rollback, deployment and incident runbooks are tested.
Documentation	OpenAPI, event catalog, schema, permission matrix, state machines and user/admin guidance are current.

ARCH-DOD-001 Traceability gate

A feature is not complete unless every requirement can be traced to code, API/schema, test evidence, release version and responsible owner.

16. Known Supersessions and Critical Risks

Area	Current authoritative direction	Risk/control
Themes	PRD_9 supersedes PRD_1 light-only direction: Platform Default, Night Mode and Creative User Type Theme.	Keep themes in presentation layer; do not fork domain behavior.
Platform support access	PRD_11 supersedes earlier time-limited/scoped rules: Company Admin grants full access until manual revocation.	Persistent warning, immediate revoke, MFA, re-authentication and immutable attribution are essential.
Public registration	PRD_6 allows professional/contractor profile applications but not automatic private company-workspace access.	Separate public identity/profile approval from company membership.
Client access	PRD_12 currently defines an internal company Client panel; PRD_8 states no direct client platform access.	Do not accidentally build a client portal until a future PRD explicitly approves it.
Client finance	PRD_13 extends PRD_12 with Finance-approved summaries and detailed payment records.	Use one Finance-owned data model; Client pages consume a permitted projection.
Project visibility	PRD_10 allows all company members to open projects while sensitive sections remain restricted.	Test nested records and section permissions; project visibility must not imply financial/HR/legal access.
Custom features	PRD_11 permits tenant-specific complete features and later promotion to core.	Use extension contracts; never create customer forks.
Super Admin Undo	Restore exact prior state where technically possible.	Classify reversibility; do not promise instant undo for unsafe operational changes.

Most serious technical failure mode

The platform will fail if every module is delivered by a different team with its own permissions, statuses, comments, documents, audit and notification logic. Governance must stop duplication before it enters production.

17. Required Deliverables from the Full-Stack Team

- C4 system, container and component diagrams covering control plane, tenant application and integration plane.
- Domain map and ownership matrix for every main entity.
- Normalized ERD plus tenant/company scoping strategy and dedicated-database migration path.
- Authorization model and executable role/section/action matrix.
- OpenAPI contracts and domain-event catalog with versioning, idempotency and ownership.
- Workflow/state-machine catalog for tasks, approvals, documents, contracts, payments, leave, support access and feature deployment.
- Design-system/component architecture showing how role-specific shells reuse shared components.
- Custom-feature extension specification, migration rules, test requirements and rollback protocol.
- Audit/timeline architecture and privileged-action reversibility classification.
- Traceability matrix mapping PRD_1-14 requirements to implementation and tests.
- Phased estimate, dependency map, staffing plan, infrastructure cost and risk register.
- Release, monitoring, backup/restore, incident response and maintenance plan.

Required team response

The team may recommend justified technical changes, but it must explicitly show how the alternative preserves tenant isolation, one source of truth, shared engines, cross-module traceability, role-specific simplicity and safe customization.

18. Acceptance Criteria

ID	Acceptance criterion
AC-14-01	The team supplies one architecture showing how every PRD uses shared identity, authorization, workflow, task, comment, document, audit, notification and search services.
AC-14-02	No business module directly reads or writes another module's private tables.
AC-14-03	Every operational request and event is tenant/company scoped and carries actor, active role and correlation context.
AC-14-04	A role-specific interface can change navigation/layout without changing the underlying permissions, statuses or record truth.
AC-14-05	A tenant-specific custom feature can be deployed, monitored and rolled back without a source-code fork or impact on another tenant.
AC-14-06	Client, contractor, contract, unit, payment, task, comment and document relationships use canonical IDs and remain cross-linked.
AC-14-07	Finance-approved manual summaries remain official while reconciliation warnings compare them with detailed payment evidence without silent overwrite.
AC-14-08	Search, exports, notifications, analytics and caches cannot reveal fields that the primary authorization engine denies.
AC-14-09	Platform support mode is explicit, fully attributed, continuously visible to the company and immediately revocable.
AC-14-10	Every Super Platform Admin Undo candidate declares exact-reversible, compensatable, backup-required or irreversible behavior.
AC-14-11	Every implemented requirement is traceable from PRD to code, schema/API, tests and release.
AC-14-12	A complete vertical workflow is demonstrated across at least two companies and multiple roles without duplicated platform logic or tenant leakage.

Final engineering statement

Construction OS becomes exceptional by keeping its large capability set while making each user experience focused and every technical rule shared. Complexity belongs in the platform architecture, not in the user interface and not in duplicated module code.